

This is excellent! Seeing that serial output means your compiler toolchain, the USB-C drivers, and the ESP32-S3 hardware are all communicating perfectly.

Since you gave me the call, **we are going to put the C++ code on the board first**. If we set up the nRF Connect app right now, you would have nothing to connect to! We need the ESP32 broadcasting its signal first.

I have recalled your two trusted MAC IDs from earlier:

- D0:D8:9C:8A:89:B4
- DE:44:73:F5:43:94

Here is **Phase 1** of your production C++ code. This script pre-loads your trusted MACs, launches the 60-second BLE Admin Server, receives commands to add/remove batteries, and then safely shuts the server down to clear memory.

Step 1: Flash the Admin Server Code

Copy this entire block, paste it into your Arduino IDE, and hit **Upload**.

C++

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <Preferences.h>

// Initialize Non-Volatile Storage (Flash Memory)
Preferences preferences;
String authorized_macs = "";

// 60-Second Timer Variables
unsigned long bootTime = 0;
const unsigned long ADMIN_TIMEOUT = 60000; // 60 seconds
bool isAdminMode = true;

// Unique Bluetooth IDs (Standard format)
#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"

// -----
// 📡 BLUETOOTH RECEIVER LOGIC
// -----

class MyCallbacks: public BLECharacteristicCallbacks {
```

```

void onWrite(BLECharacteristic *pCharacteristic) {
    // Read the text sent from the phone
    String rxValue = pCharacteristic->getValue().c_str();

    if (rxValue.length() > 0) {
        rxValue.trim();
        rxValue.toUpperCase();

        // ADD A BATTERY (+MAC)
        if (rxValue.startsWith("+")) {
            String mac = rxValue.substring(1);
            // Check if it already exists
            if (authorized_macs.indexOf(mac) == -1) {
                authorized_macs += mac + ","; // Append with comma
                preferences.putString("mac_list", authorized_macs);
                Serial.println("✅ ADDED: " + mac);
            } else {
                Serial.println("⚠️ MAC already in fleet!");
            }
        }

        // DELETE A BATTERY (-MAC)
        else if (rxValue.startsWith("-")) {
            String mac = rxValue.substring(1) + ",";
            if (authorized_macs.indexOf(mac) != -1) {
                authorized_macs.replace(mac, ""); // Remove it
                preferences.putString("mac_list", authorized_macs);
                Serial.println("🗑️ REMOVED: " + mac);
            } else {
                Serial.println("⚠️ MAC not found in fleet.");
            }
        } else {
            Serial.println("❌ Invalid Command. Use +MAC or -MAC");
        }

        Serial.println("📦 Current Fleet: " + authorized_macs);
    }
}

// -----
// ⚙️ MAIN SETUP
// -----
void setup() {

```

```

Serial.begin(115200);
delay(2000); // Give serial monitor time to open

// Open flash memory
preferences.begin("marut_app", false);

// Read saved MACs. If memory is completely empty, preload your default 2 batteries!
authorized_macs = preferences.getString("mac_list", "D0:D8:9C:8A:89:B4,DE:44:73:F5:43:94,");
preferences.putString("mac_list", authorized_macs); // Save it back just in case it's the first boot

Serial.println("\n" + String("=").substring(0, 40));
Serial.println("🚀 ESP32-S3 Admin Mode Starting...");
Serial.println("📶 Initial Fleet: " + authorized_macs);
Serial.println(String("=").substring(0, 40));

// Spin up the Bluetooth Server
BLEDevice::init("Marut_Config");
BLEServer *pServer = BLEDevice::createServer();
BLEService *pService = pServer->createService(SERVICE_UUID);

// Create the channel where data is read/written
BLECharacteristic *pCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_WRITE
);

// Attach our receiver logic from above
pCharacteristic->setCallbacks(new MyCallbacks());
pService->start();

// Start broadcasting the signal to the air
BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
pAdvertising->setScanResponse(true);
BLEDevice::startAdvertising();

bootTime = millis();
Serial.println("🌐 BLE Server Live! Connect via nRF Connect app now.");
Serial.println("⌚ 60-second timer started...");
}

// -----

```

```
// 🔄 MAIN LOOP
// -----
void loop() {
  // Check if the 60 seconds are up
  if (isAdminMode && (millis() - bootTime > ADMIN_TIMEOUT)) {
    Serial.println("\n🕒 60 Seconds up! Shutting down Admin Server...");

    // Completely kill the Bluetooth Server to free up RAM
    BLEDevice::deinit(true);
    isAdminMode = false;

    Serial.println("🔒 Switched to Flight Scanner Mode.");
    Serial.println(" (Next step: We will build the scanning logic here!)");
  }

  // Just a placeholder so the loop doesn't spin too fast after timeout
  if (!isAdminMode) {
    delay(1000);
  }
}
```

Step 2: Test it with your Phone (nRF Connect)

Once the code uploads, open your Serial Monitor. It will tell you the 60-second timer has started. You have 1 minute to do this:

1. Open the **nRF Connect** app on your phone.
2. Go to the **Scanner** tab and swipe down to refresh.
3. You will see **Marut_Config** in the list. Tap **Connect**.
4. Once connected, tap on **Unknown Service** (it will have the UUID starting with 4fafc201).
5. Under that, you will see an **Unknown Characteristic**. Tap the **Up Arrow icon** ⬆️ (This means "Write").
6. A text box will pop up. Change the format to **Text / UTF-8**.
7. Type in: **+AA:BB:CC:DD:EE:FF** and hit Send.

Look at your computer's Serial Monitor! Did you see it confirm the new MAC address and add it to your fleet list before the 60 seconds ran out?